SAE-MAS: Causal Analysis of Feedback Uptake in Solver-Critic Multi-Agent LLM Systems

Josh Schechter, Israel Omoniyi, Raye Oji

THINGS TO DO ASAP

PRIORITY:

1. Fill in the basic information for each section so we can present that
2. Essential plots and figures for explaining and visualizing the experiment and results

ESSENTIAL BUT AFTER THE ABOVE IS DONE:

1. We need examples of what single hop vs multi hop is
    a. We need to show how MuSiQue schema is set up.
    b. He has asked for this a handful of times, he surely will again
2. Examples of the QC label limitation.
    a. We need to show a few examples of where our limitations are in the QC labeling
        i. We need to show an example where the critic response is not clear or hallucinated
        ii. Show examples that are difficult for us to label. This will demonstrate that we may have to rethink how we decide to label the acceptance/rejection criteria. It may be too vague, it may not be feasible, etc
        iii. Show how we ended up having very imbalanced classes in our current labeling
        iv. Would conditions b and c actually solve any problems here not related to class imbalance?
3.

TO DO post-zoom

JOSH

- Clean up SAE slides
- Explanation & potential solution for classification problem

ISRAEL

- Diagram in draw.io for the multi-agent system, make sure to save as svg (in repo) and put up a PR
- Have an example of the MusiQue questions, different hops & answers to show
- Fill in slides
- Go through Josh's potential solution to make sure you can defend it

RAYE

- Implement steering in the causal experiment and re-run it
- Fill in slides, include new capstone diagram
- Keep updating results

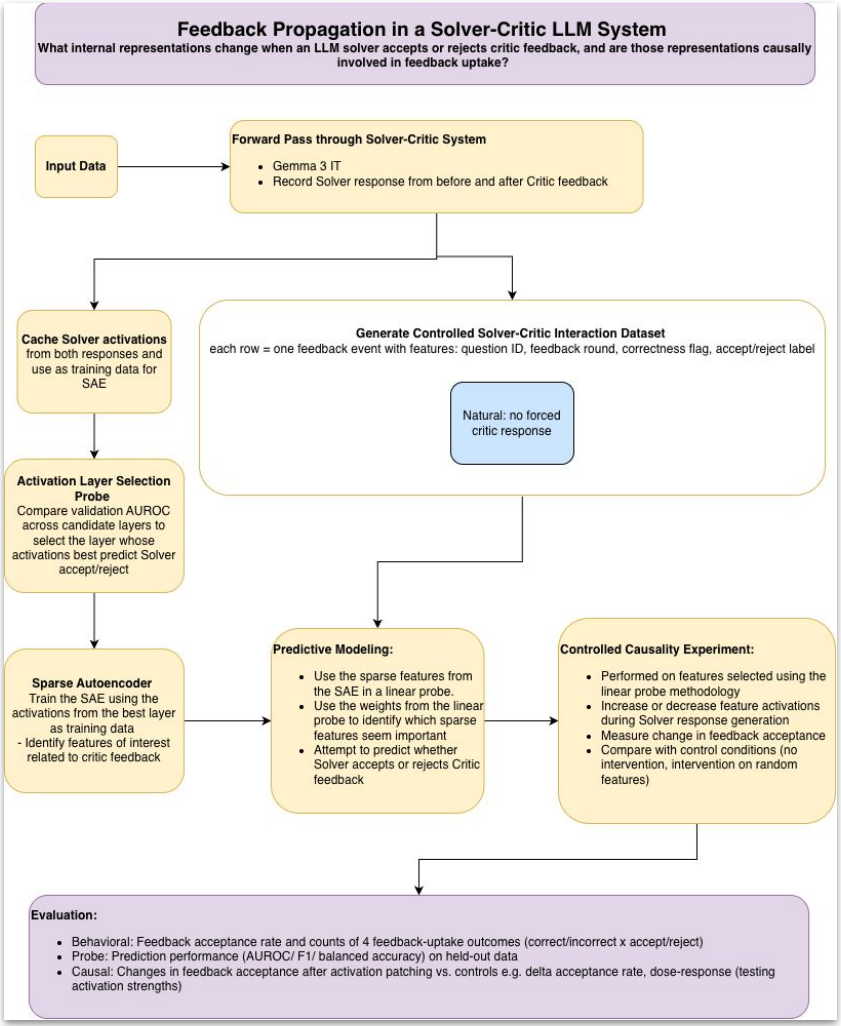
Introduction - The Research Problem

- Motivation
  - Multi-agent LLM systems are becoming increasingly relevant in modern workflows
  - An agent may adopt incorrect suggestions rather than critically evaluate them
  - Understanding why an agent accepts or rejects feedback is important for reliability of the systems
- Primary Research Question: Can SAE's prove casually faithful in multi-agent systems?
  - Can sparse autoencoders identify internal features associated with feedback adoption in multi-agent systems, and can manipulating these features causally influence the solvers behavior?
- Research Objective
  - Identify internal representations associated with feedback adoption/rejection
  - Test whether intervening on these representations changes the solvers behavior
- Potential Secondary Research Questions:
  - How effective are SAE's?
    - Can we benchmark the effectiveness of SAE's against frontier LLM's?
  - Multi-agent System Bias:
    - How can we structure Solver inputs to ensure reduced bias?

Background and Research Gap

- Mechanistic Interpretability
  - Study of how a model “thinks” internally
  - Investigates internal *activations* - can be thought of as layer outputs
  - Sparse Autoencoders (SAE) - models that are trained to decompose activations into sparse features that may be more interpretable
- Research gap
  1. Existing research provides methods for identifying and manipulating internal features, though there is only a small amount of evidence that SAE’s are effective
  2. Our research investigates whether these methods can explain and influence feedback adoption within a Solver-Critic system

Our Project Map



What a MuSiQue question actually looks like

One multi-hop question • 20 evidence/distractor passages • separate annotations

VERIFIED MUSIQUE RECORD

`id: 2hop__92495_710419`

`question:`

“Who is the spouse of the person who receive the most electoral votes in 1824?”

`paragraphs: [20 passages]`

Support #19: 1824 electoral votes → Andrew Jackson

Support #1: Andrew Jackson’s spouse → Rachel Jackson

18 other passages are distractors

MODEL INPUTS

Question + all 20 passages

DATASET ANNOTATIONS

Hop 1: Andrew Jackson

Hop 2: Rachel Jackson

Gold answer: Rachel Jackson

Each hop answers one linked sub-question

2-HOP • 15,614 QUESTIONS

“Who is the spouse of the person who received the most electoral votes in 1824?”

HOP 1

Who received most votes in 1824?

ANSWER → Andrew Jackson

HOP 2

Who is #1’s spouse?

ANSWER → Rachel Jackson ✓

3-HOP • 5,139 QUESTIONS

“Where did the spouse of the Wind Quintet composer die?”

HOP 1

Who composed Wind Quintet?

ANSWER → Carl Nielsen

HOP 2

Who is #1’s spouse?

ANSWER → Anne Marie Carl-Nielsen

HOP 3

Where did #2 die?

ANSWER → Copenhagen ✓

Four linked facts lead to one answer

A four-hop question contains four dependent lookups - not four separate test questions.

4-HOP • 1,579 QUESTIONS

“In 2018, when do we vote for the house of the body which provides oversight to the issuer of the report in 1958?”

HOP 1

Who issued the 1958 report?

ANSWER → Federal Trade Commission

→

HOP 2

Who oversees #1?

ANSWER → Congress

NEXT: HOP 3

HOP 3

Which house of #2?

ANSWER → House of Representatives

→

HOP 4

When is the 2018 vote for #3?

ANSWER → November 6, 2018 ✓

Our Natural-condition Solver → Critic → Solver

Two models, one MuSiQue question, a blind-first review, and a second Solver response.

SAME QUESTION + PASSAGES • NO GOLD ANSWER / DECOMPOSITION

SOLVER · GEMMA 3 4B

Initial answer

A1

→

CRITIC · QWEN3 4B

1. Answer blind
2. Review A1

Verdict + answer + explanation

→

SOLVER · GEMMA 3 4B

Revision after feedback

A2

WHAT WE SAVE

A1 • Critic's full feedback • A2 • outcome fields • Gemma activations

Two recorded outcomes: a correction and an ambiguity

These are saved train episodes, not invented dialogue or hypothetical examples.

EXAMPLE A • CORRECTION

“What is the area code for Cincinnati, in the state where Hotel Courtland is located?”

A1 · Solver	Critic · Qwen	A2 · Solver	Gold
234	Disagrees; recommends 513	513	513

EXAMPLE B • COHERENCE FAILURE

“The creator of Unfolding died in a city with how many in the executive branch Senate?”

A1 = “Eight” (gold: eight) → Critic advocates “2”

But the Critic’s own explanation says the answer is eight; A2 keeps “Eight.”

The current full9 run captures NINE Solver layers

Do not confuse the early nine-layer scan or old four-layer run with full9.

EARLY LAYER SCAN

2,500 episodes

9 earlier layer choices

EARLIER FULL RUN

22,332 episodes

4 layers: 8, 17, 25, 33

CURRENT FULL9

22,332 episodes

9 layers • A1 and A2

CURRENT full9 LAYERS SAVED • GEMMA HAS 34 TRANSFORMER LAYERS

L4 | L8 | L17 | L25 | L29 | L30 | L31 | L32 | L33

Each layer: 2,560-dimensional activation at the last prompt token, captured before A1 and before A2.

TRAIN	VALIDATION	TEST	INTERVENTION
15,936	2,413	1,992	1,991

Adoption dominates the automatic labels

This is a class-imbalance and measurement problem - not proof that the Solver judges feedback well.

PRIMARY TRAIN LABELS · AUTOMATIC v0.1.2



3,830 adopted (91.7%)

347 not adopted (8.3%)

14.5%

Initially wrong → finally right

2,109 / 14,527

13.1%

Critic right when A1 was wrong

1,906 / 14,527

32.3%

Initially right → finally wrong

1,233 / 3,822

347 = 111 “kept A1” + 236 “third answer.” Both groups still need feedback eligibility and semantic-match checks.

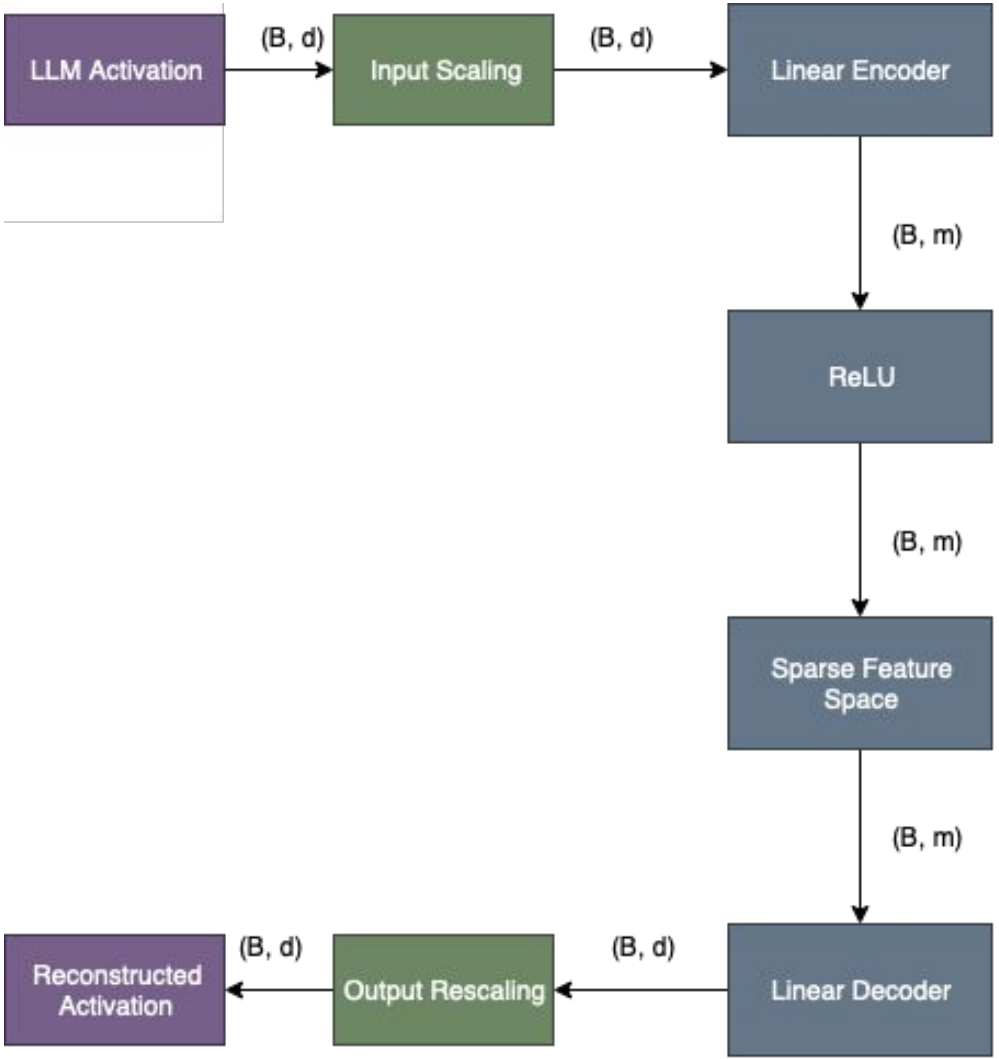
Solver (LLM) Underlying Architecture

- Talk about how each layer feeds into the next, but we don’t actually have an understanding of the semantic meaning of what the layers represent
- Explain on these layer outputs are activations
- Explain how our goal is to see if we can find semantic meaning out of these activations using SAE’s (no need to explain SAE’s yet)

Sparse Autoencoder (SAE)

- Problem it solves:
  - Superposition (polysemantic meaning): each dimension of an activation vector may represent multiple human interpretable concepts
    - We cannot assume that one activation feature has a single meaning
- SAE:
  - Encoder: Deconstruct activations into a larger, and sparse latent space
    - Creates an activation representation that is closer to monosemantic
  - Decoder: Reconstruct sparse latent space back to original activation vector
- Our goal:
  - Learn sparse representations that:
    - Reconstruct well from a sparse latent space
    - Use the dictionary broadly
    - Evaluate them in downstream interventions

SAE Architecture



Activation Collection using Solver-Critic Pipeline

- Explain how the actual activation pipeline works
- Don’t get overly technical, we really just want to explain clearly what it is doing, how the input questions work, etc.

Layer Selection Using Linear Probes

What we did

- Data: for each of 9 layers of Gemma 3 4B, we took the Solver's residual-stream activation at the final prompt token before its revised answer.

- Probe: a logistic-regression probe predicts whether the Solver will adopt the Critic's answer. It was trained on the train split, with regularization tuned by cross-validation grouped by question.

- Evaluation: on held-out validation (636 episodes: 578 adopted, 58 not adopted). We report balanced accuracy because the classes are imbalanced

- Baseline: a probe on the text alone (first answer, Critic's answer and feedback), with no activations, which tests whether the activations add anything.

Results

LAYER	BALANCED ACCURACY (95% CI)
4	0.711 (0.646–0.775)
8	0.792 (0.735–0.846)
17	0.857 (0.802–0.906)
25	0.896 (0.847–0.942)
29	0.884 (0.834–0.931)
30	0.867 (0.812–0.920)
31	0.876 (0.825–0.926)
32	0.862 (0.806–0.913)
33	0.871 (0.818–0.922)
Text only baseline	0.795 (0.732–0.855)

- Early layers (4, 8) are clearly weaker; layer 4 does worse than text alone.
- From layer 17 to 33 every layer scores 0.86–0.90, and none is statistically separated from the best one, layer 25.
- Layer 25 is about 10 points above the text-only baseline (CI excludes 0), so the activations carry signal beyond the visible text.
- Choice: we started with layer 33, the model's final layer and the one closest to the output. We then added layer 25, which had the highest observed score.

SAE Section Plan

1. Background:
   a. draw.io diagram (svg or pdf output) for SAE architecture
   b. Explain what an SAE is useful for and why we are using it in the context of our project
2. L1 training
   a. $\mathcal{L} = \mathcal{L}\text{reconstruction} + \lambda \text{ mean}(|z|)$
   b. Explain Reconstruction loss
      i. Measurement of how well the SAE can reconstruct the original activation vector after deconstructing it to sparse latent space
   c. Explain Sparsity Penalty
      i. Encourages the SAE to deconstruct the activation into a latent space that has fewer activated features. This is important because it encourages a latent feature space that only activates when theoretically a certain semantic meaning is part of the token
3. Top k training
   a. $\mathcal{L} = \mathcal{L}\text{reconstruction} + \alpha\, \mathcal{L}\text{AuxK}$
   b. Explain AuxK
      i. $\mathcal{L}$AuxK = MSE(inactive-feature prediction, normalized reconstruction error)
      ii. α = aux coefficient
   c. AuxK
      i. An auxiliary training loss that helps inactive features learn from the remaining reconstruction error
         1. Idea was to give opportunities for features that were never being selected by top k to learn
         2. Without this, we would just apply top K to the features it selects, and hope that it happens to select a diverse amount of features to increase dictionary usage
4. Results
   a. Show the plot
   b. Explain why I selected each model from each training strategy

SAE Training

- For now, trained an SAE for layers 25 and 33 of the gemma model (our Solver)
- Have models trained on 2 different approaches to encourage sparsity:
  - L1: sparsity coefficient
  - L0: top K feature activation
- We tuned both and selected the most interesting/promising of each strategy in the downstream causality pipeline
  - This was done because we wanted to see if one strategy produces a better model for our interpretability task
- Factors to account for:
  - Reconstruction accuracy
  - Sparsity
  - Dictionary usage

SAE Training: Sparsity Coefficient (L1)

- Training task: Deconstruct activations into sparse features, then reconstruct to original activations
  - Training must account for:
    1. Reconstruction error
    2. Sparsity
- Reconstruction Loss:
  - Compute the error of the models reconstruction compared to that of the original activation
- Sparsity Coefficient (Penalty)
  - Encourages the SAE to use as few latent features as possible when reconstructing the activations
- One more thing to consider: Dictionary usage
  - During training, the model shouldn't use the same features to reconstruct everytime, otherwise it may not actually help with our interpretability task

- Loss function
  - $\mathcal{L} = \mathcal{L}\text{reconstruction} + \lambda \text{ mean}(|z|)$
    - λ = sparsity coefficient
    - z = latent activations

SAE Training: Top K (L0) + Aux K

- Top K
  - Only allow top k features to be active on a given sweep -> this forces the active features to as sparse as k
  - Problem: what if during training, the model learns to reconstruct from the same k features?
  - Solution: AuxK method, where we have an auxiliary loss function that helps encourage the model to use other features that are not being used in the top k in order to give them a chance to learn

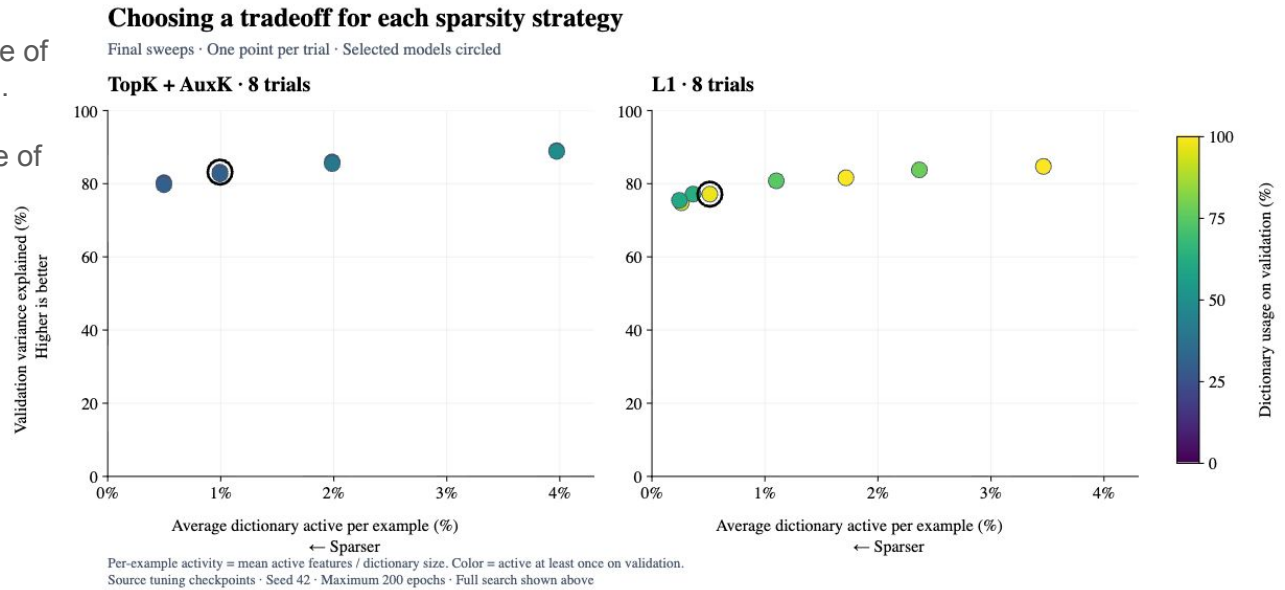
- Loss function:
  - $\mathcal{L} = \mathcal{L}\text{reconstruction} + \alpha\, \mathcal{L}\text{AuxK}$
    - α = auxiliary coefficient
    - $\mathcal{L}$AuxK = reconstruction loss encouraging inactive features to learn representations

SAE Results

- 3 metrics of interest:
  1. Good reconstruction accuracy (Variance Explained)
     a. VE = 1 - (MSE SAE / MSE Baseline)
     b. Baseline averages each dimensions value (from activations) and uses that as residual activation for dimensions
  2. High percentage of dictionary usage
     a. If the SAE learns to reconstruct well, but uses the same k features to do so every time, that does not give us a useful latent space for interpretability
  3. Sparse latent features
     a. Identify the smallest amount of generally active features while still maintaining reasonable reconstruction score and high dictionary usage
- Model selection accounts for all of these

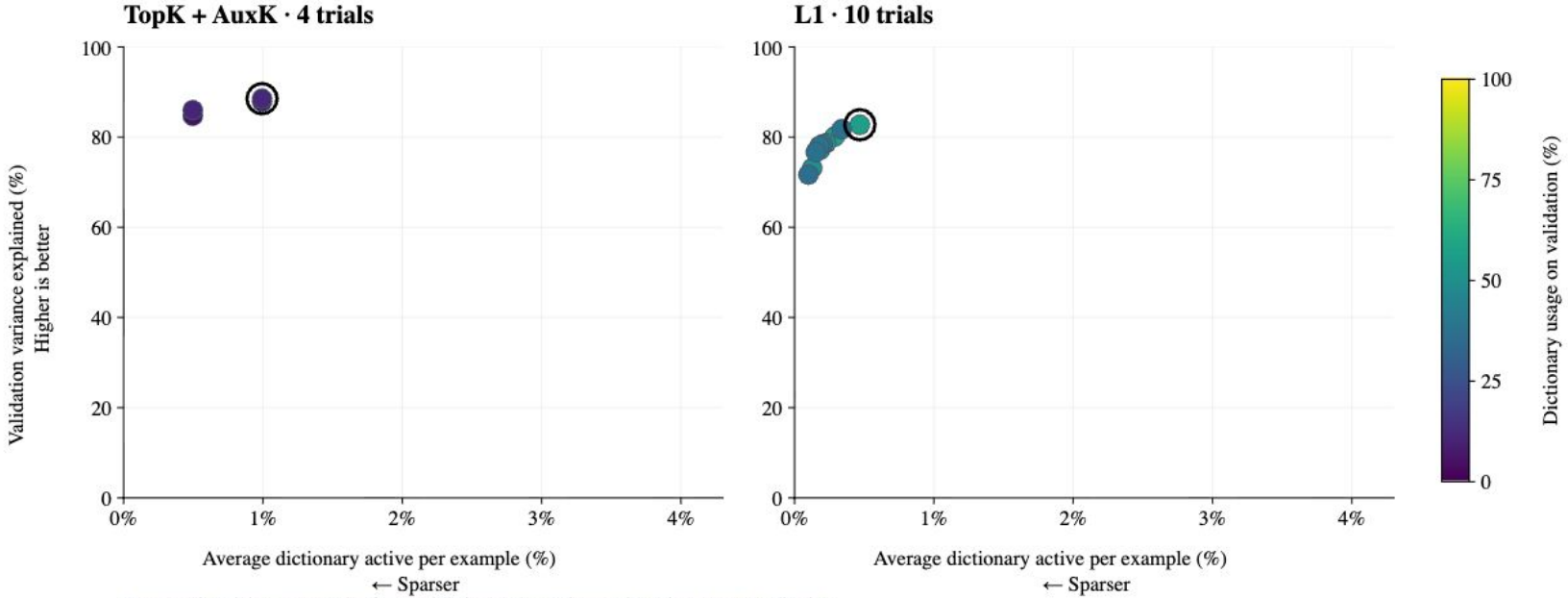
SAE Model Selection

- **Reconstruction quality (Y-axis)**: Validation variance explained measures how much the SAE reduces reconstruction error versus predicting the training average. Higher is better.
- **Sparsity (X-axis)**: Average percentage of dictionary features active per example. Lower means sparser.
- **Dictionary usage (color)**: Percentage of features active at least once across validation examples. Higher means broader dictionary coverage.



Layer 25 · Choosing a tradeoff for each sparsity strategy

Source sweeps · One point per trial · Selected models circled



Per-example activity = mean active features / dictionary size. Color = active at least once on validation.

Source tuning checkpoints · Seed 42 · Maximum 200 epochs · Full search shown above

Feature Selection Using Linear Probes

What we did

- Encoded the Solver's layer-25 & layer-33 activations with three SAEs: our TopK SAE, our L1 SAE, and Google's pretrained Gemma Scope SAE.

- Trained a class-weighted logistic-regression probe on the SAE features to predict whether the Solver adopts the Critic's answer (train n = 4,177, validation n = 636).

- Ranked features by mean |SHAP| on the training set and selected the top 10 features that push toward adopting.

- Probe readout: zeroed those features in the validation activations and measured how much a probe trained on the raw activations changes its predicted acceptance, compared with zeroing 10 random features (50 draws).

Layer 25 results

SAE	Probe balanced accuracy (95% CI)	Probe predicted acceptance, no ablation	Δ, top-10 zeroed	Δ, 10 random zeroed
TopK	0.76 [0.70, 0.83]	90.9%	−14.8 pp	+0.2 pp
L1	0.73 [0.67, 0.80]	91.9%	−8.2 pp	−0.2 pp
Pretrained	0.78 [0.71, 0.85]	92.4%	−1.8 pp	−0.4 pp

Δ = change in probe-predicted acceptance, in percentage points (pp), relative to that SAE's no-ablation row.

- SAE features encode the acceptance decision well above chance (balanced accuracy 0.73–0.78).
- The selected features carry the probe's signal: zeroing them moves probe-predicted acceptance far more than zeroing random features, most strongly for TopK.

Layer 33 results

SAE	Probe balanced accuracy (95% CI)	Probe predicted acceptance, no ablation	Δ, top-10 zeroed	Δ, 10 random zeroed
TopK	0.76 [0.70, 0.83]	90.9%	−14.8 pp	+0.2 pp
L1	0.73 [0.67, 0.80]	91.9%	−8.2 pp	−0.2 pp
Pretrained	0.78 [0.71, 0.85]	92.4%	−1.8 pp	−0.4 pp

Δ = change in probe-predicted acceptance, in percentage points (pp), relative to that SAE's no-ablation row.

- SAE features encode the acceptance decision well above chance (balanced accuracy 0.73–0.78).
- The selected features carry the probe's signal: zeroing them moves probe-predicted acceptance far more than zeroing random features, most strongly for TopK.

Casual Intervention Experiment

- Go into detail about how this pipeline was set up
- Display the results in tables, plots, diagrams, etc.

Causality Experiment Initial Results

Experiment	Condition	Acceptance Rate (%)	Δ .vs. SAE Control (pp)	95% CI for Δ	p-value
Baseline	No SAE				
TopK SAE	No ablation				
	Random ablation				
	Targeted ablation				
L1 SAE	No ablation				
	Random ablation				
	Targeted ablation				
Pretrained SAE	No ablation				
	Random ablation				
	Targeted ablation				

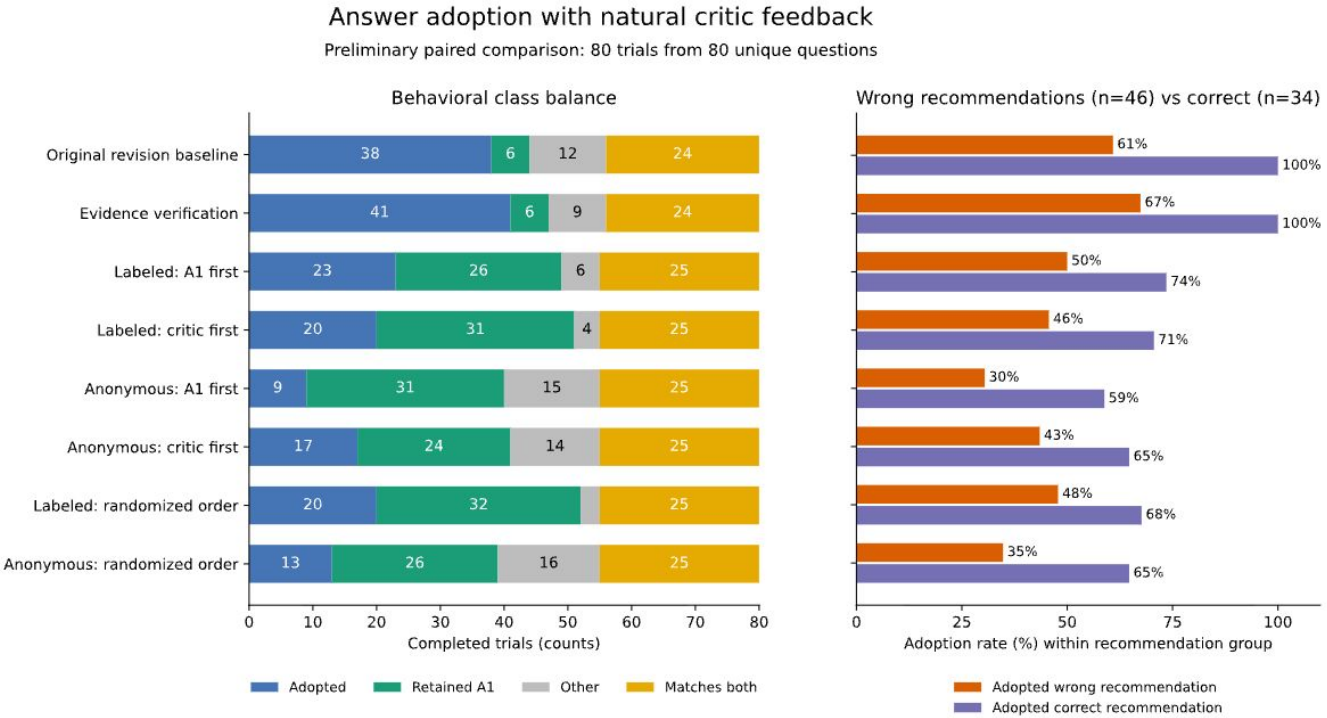
Limitations

- Small LLM’s (compute capacity)
- Trained SAE’s do not have a loss of 0
- ****Solver overwhelming defaults to adopting critic feedback****

The Solver Adoption Problem

- The solver was initially changing its answer to whatever the critics suggestion was for nearly all questions
  - Creates class balancing issue: linear probes are trained on primarily one case
- Hypothesis:
  - When the solver gets feedback, it only receives the critics suggestion. This creates a bias. If we expose the solver to both its original answer, and the critics, it could result in a more balanced distribution of adoption
- Results:
  - Giving the Solver an option between its original answer and the Critic’s feedback yielded a far more balanced adoption distribution

Solver Feedback Adoption Experiment



Same 80 questions as the fixed-feedback plot; one natural recommendation per question, shared across protocols.
51 saved recommendations; 29 regenerated. Original/verification see full feedback; candidates see the answer only.
Matches both is ambiguous adoption/retention. Automatic alias-aware matching; selected development sample.

Next Steps

1. Re-run activation collection, this time giving the Solver an option between its original answer and the Critics suggestion
2. Continue training SAE's
    a. If l1 continues to outperform, consider training the decoder to reconstruct better with a frozen encoder to keep latent sparsity and usage
    b. For top k: improve dictionary usage to give a more fair comparison to L1 trained method
3. Benchmark SAE interpretability against a frontier LLM
    a. Frontier model can take in not only activations, but other context as well
    b. While causality would be hard to prove, we could test for correlation